Buffalo Accident Risk Prediction & Resource Allocation
An Application of Reinforcement Learning for Emergency Response Optimization

Authors: Meghna Trichur Saravana Shekhar, Sahil Shivaji Sawant, Fagun Nirag Patel
Department of Computer Science & Engineering
School of Engineering & Applied Sciences, University at Buffalo

Abstract

This project presents a reinforcement learning (RL) approach to optimize emergency resource allocation in Buffalo, NY. By analyzing accident data and applying the Deep Q network Proximal Policy Optimization (PPO) algorithm, we created an intelligent system that can determine the optimal placement of emergency services to minimize response times across different accident types. This system allocates resources based on predicted risk patterns derived from historical accident data, significantly outperforming random allocation behaviors.

1. Introduction

Traffic accidents in urban areas present a critical public safety challenge, with Buffalo recording approximately 10,000 incidents annually. Traditional approaches to emergency resource allocation are often unable to adapt to the nature of accident occurrence patterns. This project addresses this problem by developing a reinforcement learning framework that dynamically allocates emergency resources based on real-time risk assessment.

The goals of this project are to:

1. Analyze historical accident data to identify risk patterns across Buffalo
2. Develop a reinforcement learning model that optimizes emergency resource allocation
3. Compare the performance of our RL-based approach
4. Provide insights for potential real-world implementation

2. Data Analysis

2.1 Data Source

The project utilizes the "Received Traffic Incident Calls" dataset from the City of Buffalo's open data portal (Link). It is a real world dataset. This dataset contains detailed information about traffic incidents reported to emergency services, including:

- Location coordinates (latitude and longitude) in POINT format
- Accident type
- Neighborhood and ZIP code

2.2 Preprocessing

We processed the raw data to extract location information from the POINT format and filtered incidents to focus on Buffalo city limits. The data is categorized into three main accident types:

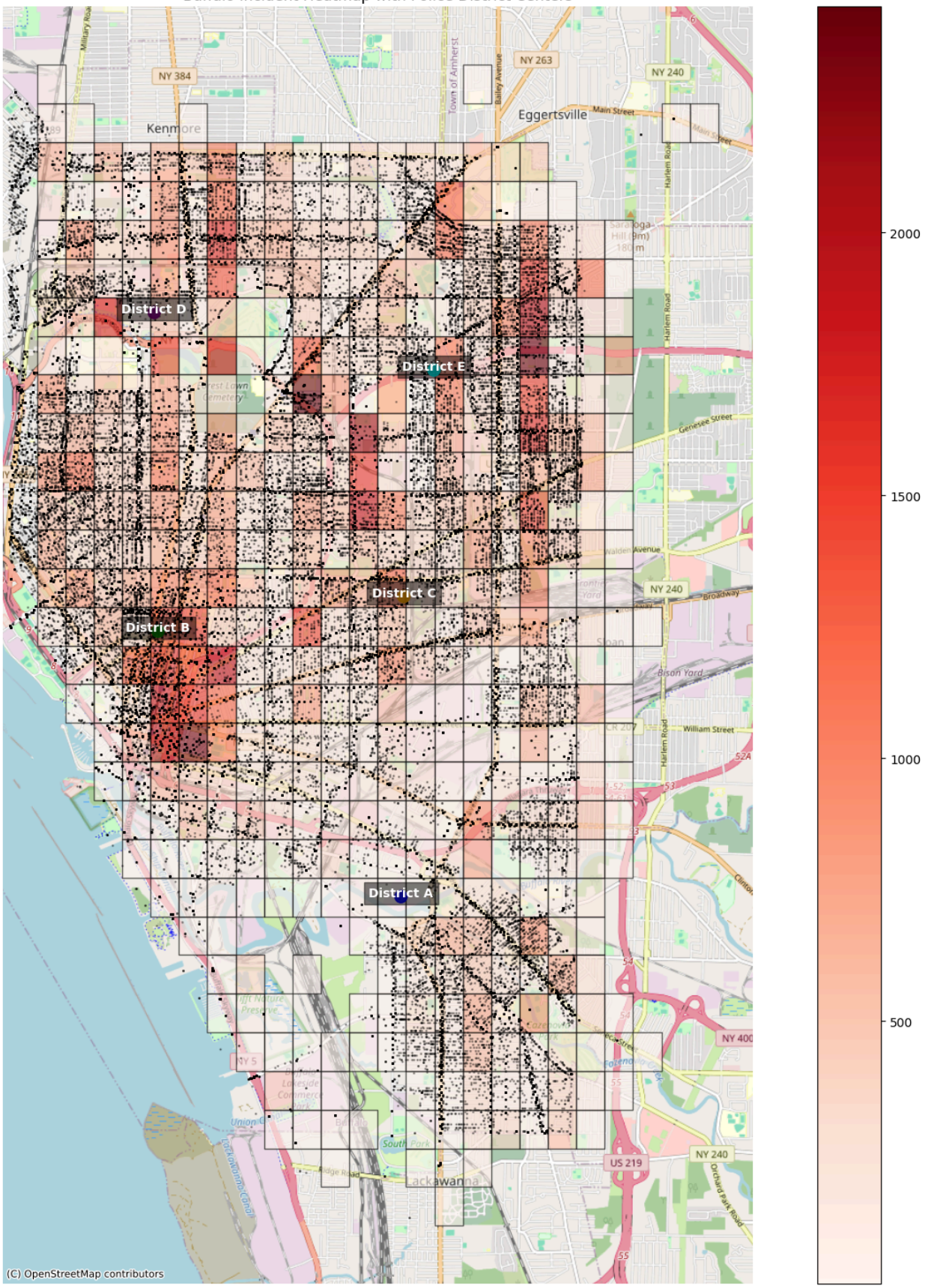
1. Accident Property Damage Only
2. Accident/Injury
3. Accident Skyway/33/198

We also removed the null values as they were only 3% of the total dataset.

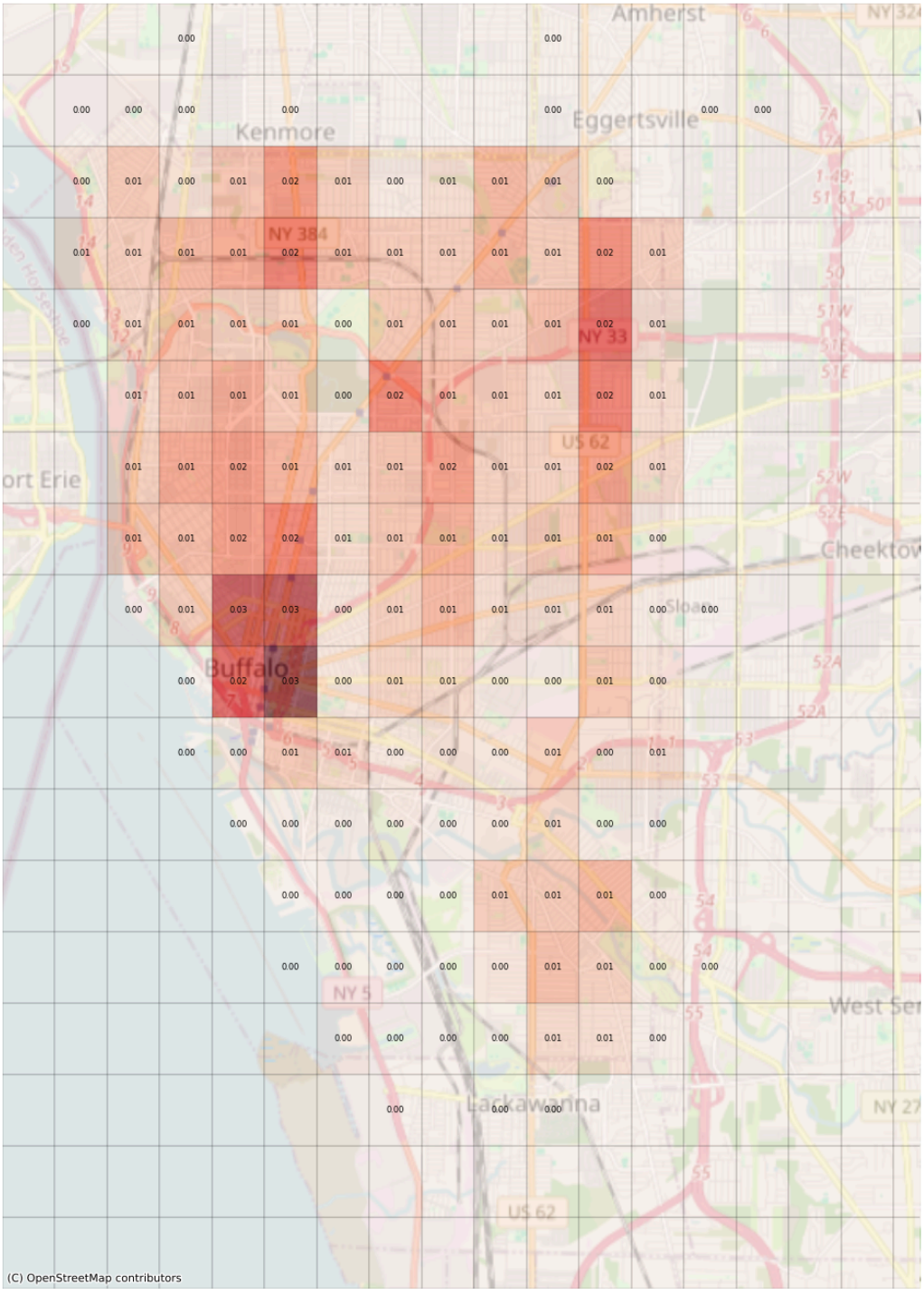
2.3 Data Analysis

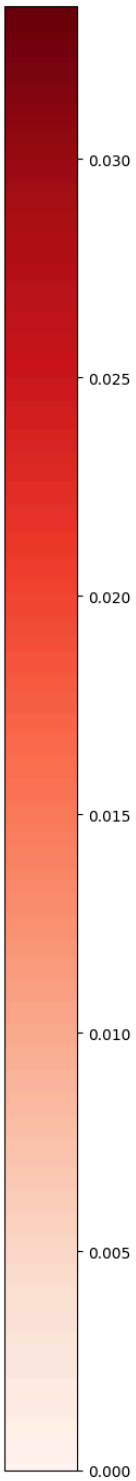
The city was divided into a grid where each cell represents approximately a 500m × 500m area. For each grid cell, we calculated the probability of each accident type occurring based on historical data. This analysis revealed distinct patterns. On an average, 404 incidents were reported in each grid. A total of 145 grids reported various incidents.

Buffalo Incident Heatmap with Police District Centers

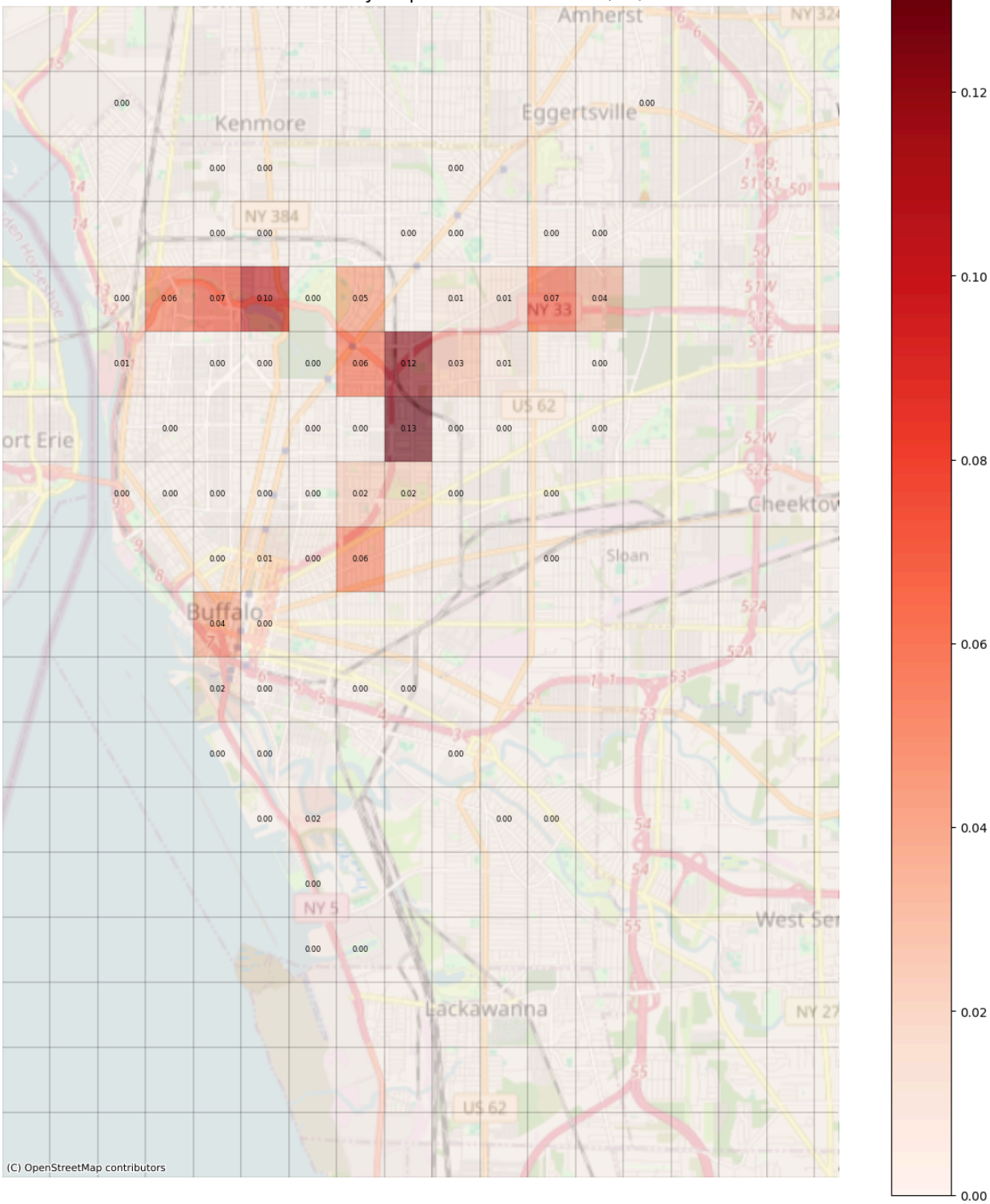


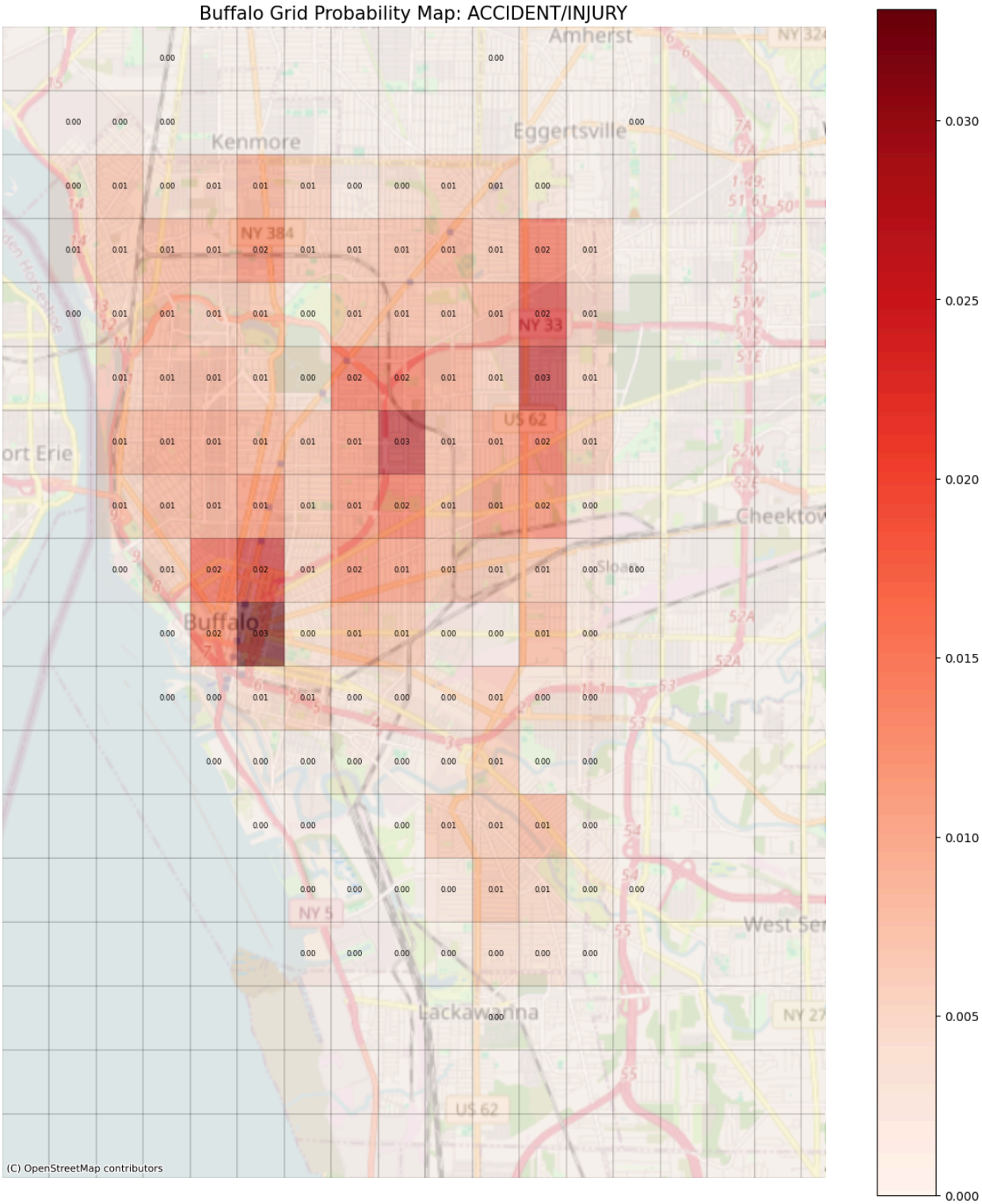
Buffalo Grid Probability Map: ACCIDENT PROPERTY DAMAGE ONLY





Buffalo Grid Probability Map: ACCIDENT SKYWAY/33/198




Buffalo Grid Probability Map: ACCIDENT/INJURY
Amherst
Kenmore
Eggertsville
Buffalo
Lackawanna
NY 384
NY 33
US 62
NY 5
0.030
0.025
0.020
0.015
0.010
0.005
0.000
(C) OpenStreetMap contributors

- **Property Damage Only accidents**: More evenly distributed throughout the city with concentration in downtown areas
- **Injury accidents**: Similar pattern to property damage but with slightly higher concentration in specific areas
- **Skyway/33/198 accidents**: Highly concentrated along major highways with specific hotspots showing probability values up to 0.13 (13%)

The probability maps clearly identify high-risk zones that should be prioritized for emergency resource allocation.

3 . Reinforcement Learning Approach

3.1 Problem Formulation

We formulated the resource allocation problem as a Markov Decision Process (MDP):

- **State**: A comprehensive vector representing:
  - Accident probabilities across all grids for each accident type
  - Total available resources by type
  - Current allocation of resources by type and grid cell
  - Remaining resources by type
  - Cell resource limits by type
- **Actions**: Three-part actions consisting of:
  - Action type (0: allocate, 1: deallocate)
  - Resource type index (police, ambulance (EMS), tow truck (DOT) (DOT))
  - Target grid cell index
- **Reward**: reward function that incentivizes:
  - Meeting resource needs in medium and high-need areas (large positive reward)
  - Penalties for unmet needs (scaled by need level)
  - Small penalties for allocating resources to low-need areas
  - Penalties for unused resources
  - Action-specific penalties (invalid allocation, deallocation, etc.)
- **Environment**: Buffalo city map represented as a grid with accident probabilities derived from historical data and transformed into need levels (0: low, 1: medium, 2: high) for each accident type

3.2 Proximal Policy Optimization (PPO)

We chose PPO as our reinforcement learning algorithm due to its stability and performance on continuous control tasks. PPO uses a clipped objective function to prevent excessive policy updates

3.3 Environment Design

Our implementation used a custom OpenAI Gym environment

(ResourceAllocationEnv) with several key components:

Resource Mapping

Resources are mapped to specific accident types:

- ACCIDENT/INJURY: Requires police and ambulance (EMS)
- ACCIDENT PROPERTY DAMAGE ONLY: Requires police and tow truck (DOT) (DOT)
- ACCIDENT SKYWAY/33/198: Requires police, ambulance (EMS), and tow truck (DOT) (DOT)

Need Levels

Accident probabilities are converted to discrete need levels:

- Low (0): Below the medium threshold
- Medium (1): Between medium and high thresholds
- High (2): Above the high threshold

The thresholds vary by accident type:

- ACCIDENT/INJURY: (0.01, 0.02)
- ACCIDENT PROPERTY DAMAGE ONLY: (0.01, 0.02)
- ACCIDENT SKYWAY/33/198: (0.03, 0.09)

Reward Structure

Several reward structures were designed to solve this complex environment which were tested against different algorithms. These reward structures are explained below.

Structure 1

Positive Rewards

1. Meeting Needs (met_need_reward): When all required resources for a high-risk accident type are allocated to a grid with medium or high need, the agent receives a positive reward.
   This reward is scaled based on the need level:
   1. Medium need: multiplied by 1.5

2. High need: multiplied by 3.0
This encourages the agent to prioritize and completely fulfill high-need areas.

Penalties

1. Unmet Needs (unmet_need_penalty): If any of the required resources for a needed accident type are missing in a medium/high-need cell, the agent is penalized.
   Like the reward, the penalty is also scaled with need level (medium = 1.5, high = 3.0).

2. Wrong Allocations (wrong_allocation_penalty):
   If resources are placed in cells that have only low need across all accident types, a small penalty is applied for each resource type unnecessarily placed.
   This discourages wasteful deployment to low-risk zones.

3. Unused Resources (unused_resource_penalty_factor):
   A small penalty is applied proportionally to the number of resources left unused.
   This gently pushes the agent to utilize available resources over time instead of idling them.

Invalid Actions:

Several high penalties are enforced to discourage invalid or inefficient actions:

- Over-allocating beyond cell limits = -2.0
- Allocating when no units are left = -2.0
- Deallocating from a cell where nothing is assigned = -0.5
- Incorrect action input = -5.0

Structure 2:

The reward function was carefully designed to encourage optimal resource allocation:

```
default_reward_config = {
   # Make meeting needs highly rewarding
   "met_need_reward": 1000.0, # Base reward
   "met_need_factor": {0: 0.0, 1: 1.5, 2: 3.0}, # Strong scaling for higher need

   # Keep unmet need penalty significant but maybe less than max reward
   "unmet_need_penalty": -5.0, # Base penalty
   "unmet_need_factor": {0: 0.0, 1: 1.5, 2: 3.0}, # Strong scaling

   # Drastically reduce penalty for placing in low-need cell
```

```
    "wrong_allocation_penalty": -0.1, # Penalty per resource *type* in low-need cell

    # Penalty for unused resources (NEW) - Make it small per resource
    "unused_resource_penalty_factor": -5.0, # Penalty per *unit* of unused resource

    # Keep penalties for invalid actions relatively high
    "over_allocation_action_penalty": -2.0,
    "no_resource_action_penalty": -20.0,
    "dealloc_nonexistent_penalty": -0.5,
    "invalid_action_structure_penalty": -5.0,
}
```

Compared to the earlier configuration, the updated reward structure introduces more aggressive incentives and penalties:

- The reward for meeting needs (met_need_reward) is increased from 10.0 to 1000.0, significantly amplifying the positive feedback for correct resource allocations in high-need zones.

- The penalty for unused resources (unused_resource_penalty_factor) is raised from a -0.01 to a much higher -5.0 per unit, strongly discouraging idle resources.

- The penalty for attempting to allocate with no resources available (no_resource_action_penalty) jumps from -2.0 to -20.0, making such invalid actions far more costly.

Other components like scaling factors, wrong allocation penalty, and structure remain unchanged

Structure 3:

Positive Reward Component
Need Matching Reward

- match_reward = sum(resource_need_prob * current_allocation)

- This gives a positive reward for placing resources in zones that have high probability of needing them.

- It's proportional: allocating more resources to high-need zones gives higher rewards.

- Example: if a grid has high probability of a fire, allocating more fire units there will give a bigger reward.

Penalty Components

Movement Cost

- movement_cost = sum(abs(current_allocation - previous_allocation))
- Penalizes the agent for changing the allocation too much between steps.
- Prevents unnecessary shuffling of resources and encourages stable, minimal-change policies.
- Weighted by movement_cost_factor (e.g., 0.08).
-

Over-Allocation Penalty

- If the total requested resources exceed the available count, the agent is penalized:

over_alloc_penalty = (requested - available) * 0.05

- This keeps the agent from overpromising resources during allocation steps.

Final Reward Calculation
At each step, the reward is:
Reward = match_reward − (movement_cost × movement_cost_factor) − over_alloc_penalty

Structure 4: USING SET OF RESOURCES

1. Reward for Correct Allocation (Full Reward)

- When the agent allocates the exact resource set required by an incident type in a grid cell (e.g., [1, 1, 1] for SKYWAY), it receives a high positive reward.

2. Reward for Partial Allocation (Partial Reward)

- If the agent allocates a different set of resources than what is required at a grid cell, it still receives a smaller reward.
- This encourages the agent to act even if it cannot fully satisfy the demand, rather than taking no action.

3. Penalty for No Allocation

- A significant penalty is applied to any grid cell that required a resource set but received no allocation.
- This promotes responsiveness to critical zones and avoids agent inactivity.

4. Unused Resource Penalty

- At the end of each timestep, the agent is penalized for any unallocated resources remaining from the initial total pool.

- For example, if 30 police, 37 ambulance (EMS)s, and 38 tow truck (DOT) (DOT)s remain unallocated from a total of [50, 50, 50], the agent is penalized for those 105 unused resources. This incentivizes proactive deployment.

5. Over-Allocation Penalty

- Each grid cell has a maximum limit for each resource type (e.g., 3 police, 3 ambulance (EMS)s, 3 tow truck (DOT) (DOT)s).

- If the agent allocates resources to a cell exceeding these limits, it receives a penalty for that allocation instead of a reward.

6. Invalid Allocation Penalty

- If the agent attempts to allocate more resources than are available in the total pool (e.g., allocating police when only 0 are available), it is penalized for performing an invalid action.

7. Inactivity Penalty

- The agent receives a penalty for choosing to take no action in a timestep. This discourages idle behavior.

8. Invalid Deallocation Penalty

- If the agent tries to deallocate a resource set from a cell that has no such resources, it receives a penalty.

- This ensures deallocation actions are performed only where previously allocated resources exist.

3.4 Network Architecture

Our implementation used an Actor-Critic architecture:

- **Actor Network**: A Multi-Layer Perceptron (MLP) that outputs the policy (action probabilities)
- **Critic Network**: A parallel MLP that estimates the value function for each state

Both networks process the state vector comprising accident probabilities, resource allocation status, and resource availability information.

3.5 Training Process

The PPO agent was trained over multiple episodes, with each episode allowing up to 500 steps of resource allocation decisions. During training, the agent learned to:

1. Recognize patterns in accident probabilities across different areas
2. Identify which grid cells had medium or high need levels for different accident types
3. Allocate appropriate resource combinations based on accident type requirements (police, ambulance (EMS), tow truck (DOT) (DOT))
4. Make efficient use of limited resources to maximize coverage of high-need areas
5. Avoid wasting resources in low-need areas
6. Balance the trade-off between meeting current needs and maintaining resources for future allocation

4. Results

4.1 Performance Metrics

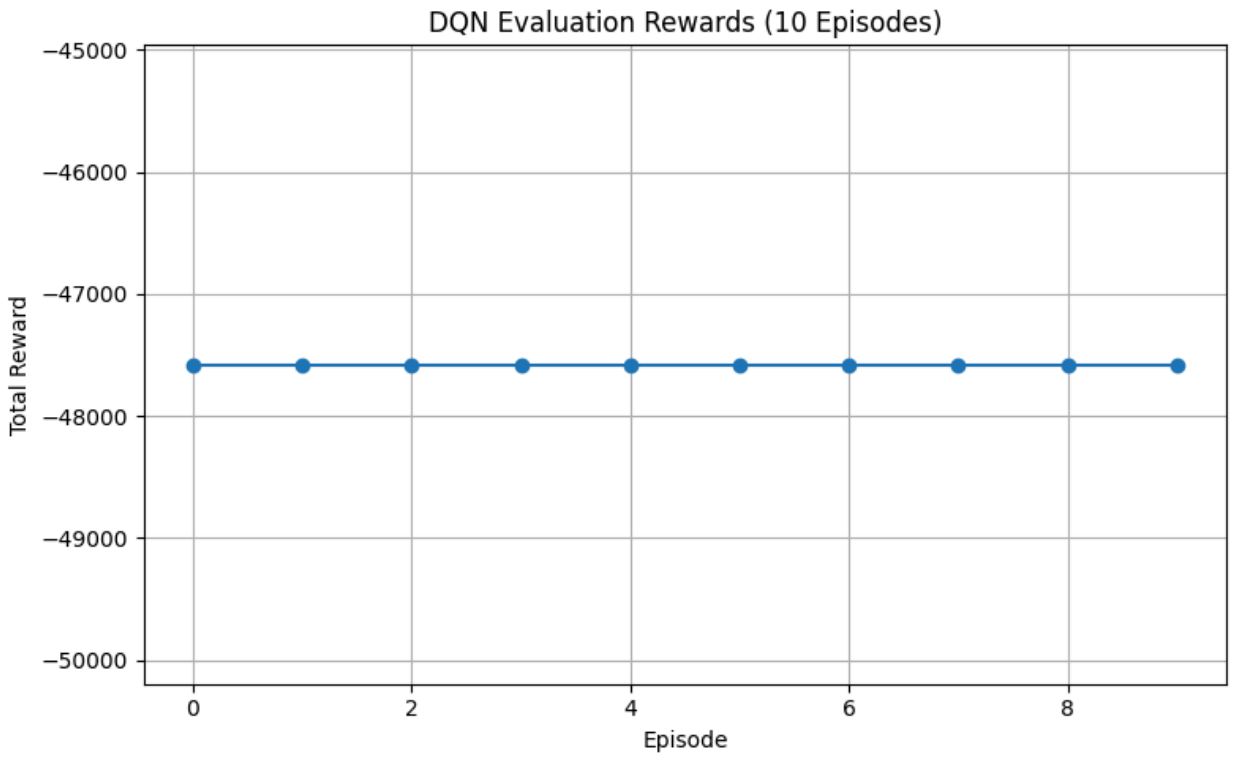
We evaluated our model using the following metrics:

- **Average reward**: Reflecting the negative expected response time
- **Resource utilization efficiency**: Measuring how effectively resources were distributed
- **Coverage of high-risk areas**: Assessing whether the model correctly prioritizes areas with higher accident probabilities

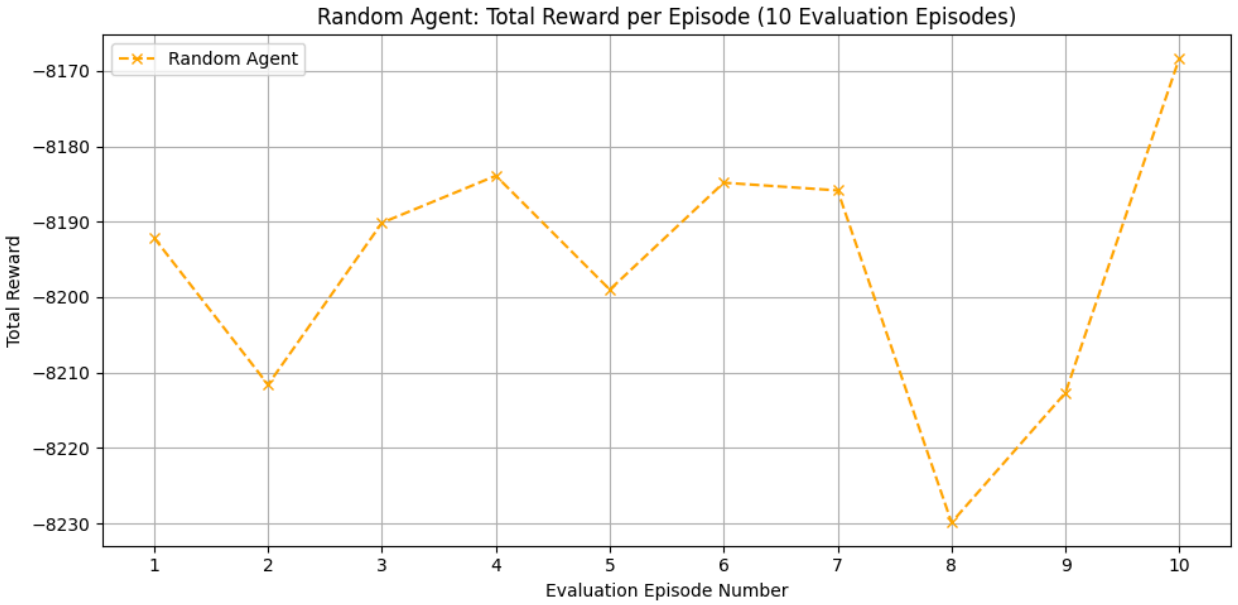
4.2 Comparison study

We compared our PPO agent against a random allocation baseline that distributes resources randomly across the grid. The results demonstrate the significant improvement achieved by the RL approach:

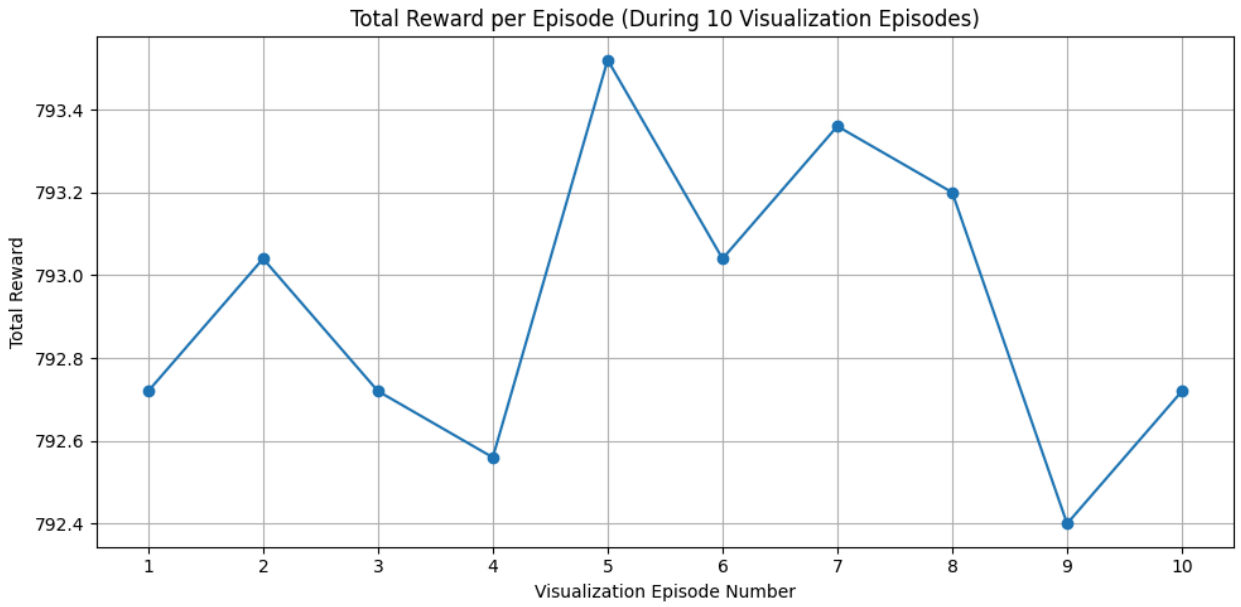
- DQN Agent:



- The DQN agent performed poorly on the previous reward structures wherein the agent was hesitating to allocate resources as it was being heavily penalised due to the reward strategy
- **Random Agent**: Rewards ranged between -8170 and -8230

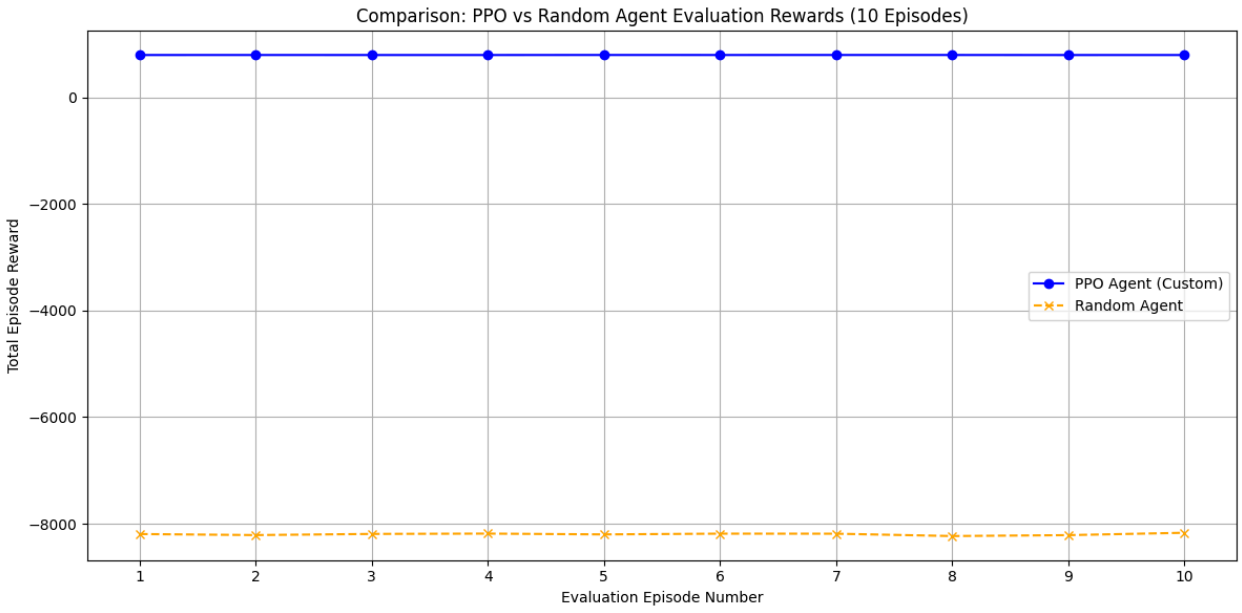


- **PPO Agent**: Consistently achieved a reward of around 793 across 10 evaluation episodes



This massive performance gap (several orders of magnitude) demonstrates that the PPO agent learned effective allocation strategies. The random agent frequently:

- Placed resources in low-need areas (incurring wrong allocation penalties)
- Failed to meet needs in high-priority areas (missing the substantial met_need_reward)
- Made invalid allocation attempts (incurring action penalties)
- Left resources unused (receiving unused resource penalties)



In contrast, the PPO agent demonstrated strategic behavior, placing complementary resource types together (police, ambulance (EMS), tow truck (DOT) (DOT)) in high-need areas to receive the substantial reward for meeting needs, particularly in high-need cells where the reward was amplified by the need factor of 3.0.

PPO Implementation

The implementation used PyTorch for the neural network components and OpenAI Gym for the reinforcement learning environment. Key hyperparameters included:

```
ppo_config = {
    "learning_rate": 3e-4,
    "n_steps": 2048,
    "batch_size": 64,
    "n_epochs": 10,
    "gamma": 0.99,
    "gae_lambda": 0.95,
    "clip_epsilon": 0.2,
    "ent_coef": 0.01,
    "vf_coef": 0.5,
    "max_grad_norm": 0.5,
    "device": DEVICE
```

4.3 Analysis of Allocation Patterns

The trained PPO agent demonstrated intelligent allocation behavior:

- Resources were predominantly allocated to areas with higher accident probabilities
- The agent learned to differentiate between different accident types and their spatial distributions
- Resource allocation changed dynamically based on temporal patterns in the data
- The agent maintained an appropriate balance between coverage and response time optimization

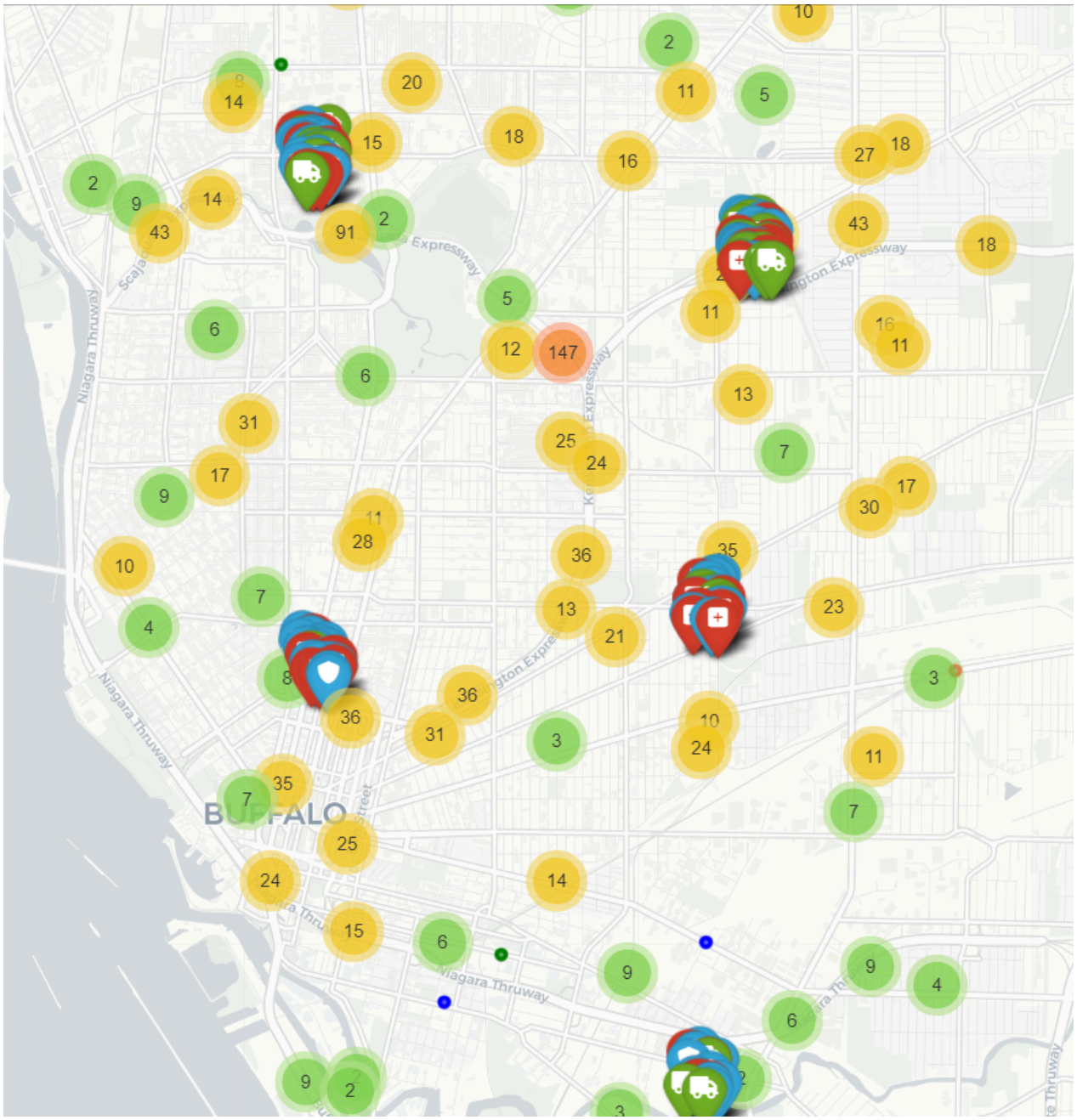
Discussion

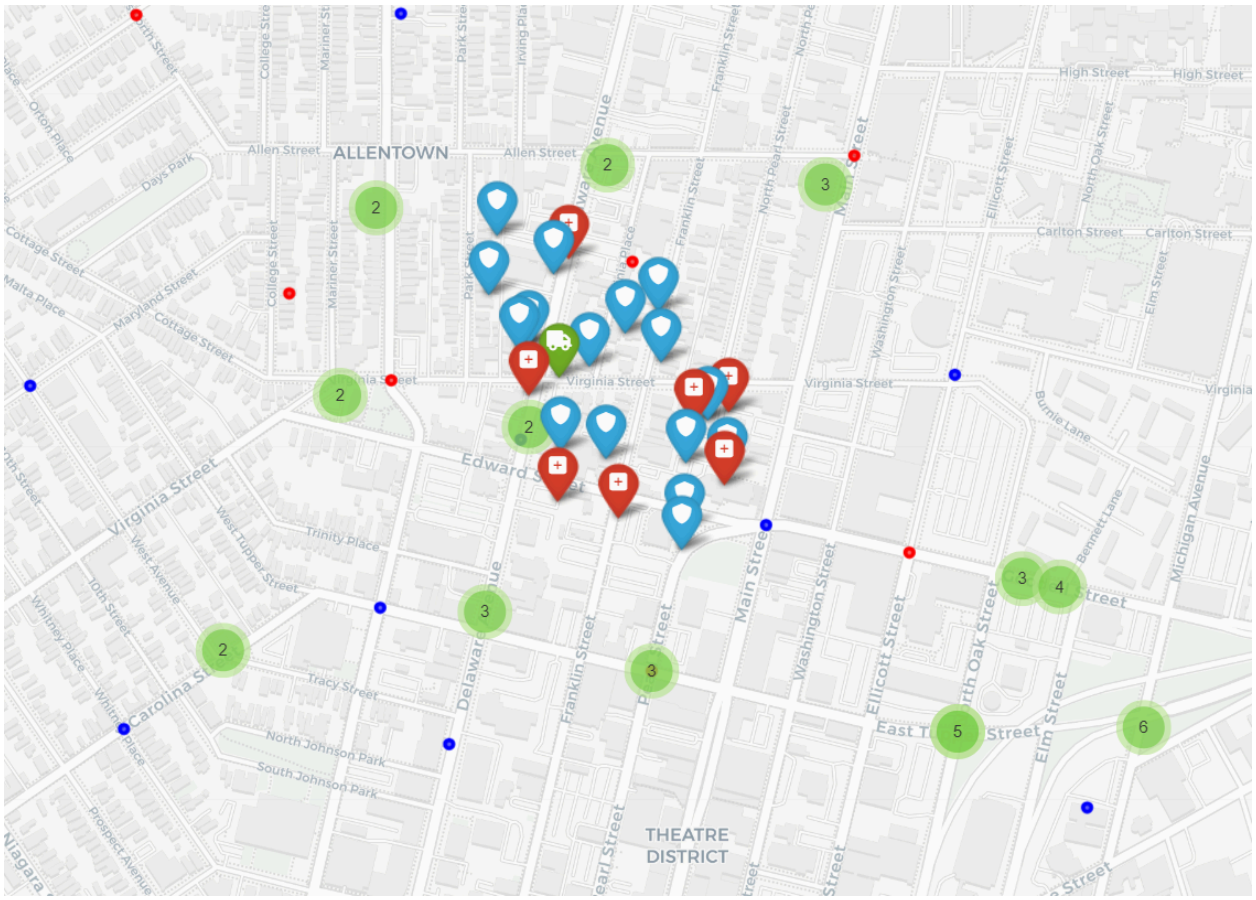
5.1 Key Insights

Our results demonstrate that reinforcement learning can effectively optimize emergency resource allocation in urban environments. The PPO agent learned to:

- Recognize the importance of placing resources near high-probability accident locations
- Balance the trade-off between concentrating resources in hotspots versus maintaining adequate coverage
- Adapt resource allocation based on the specific characteristics of different accident types

5.1 Visualizations using interactive map





Above graph shows clustering of various accidents and resources across the Buffalo Map

5.2 Limitations

Several limitations should be acknowledged:

- Our model assumes a static probability distribution throughout the evaluation period from the existing dataset
- The grid representation may not perfectly capture the complex road network of Buffalo
- Limited resources in the simulation may not reflect actual emergency service capacity
- Need of stronger reward structure that captures several other dependencies for better motivation to learn allocation and deallocation

5.3 Future Work

Future improvements could include:

- Incorporating variations in accident probabilities (time of day, day of week, seasonal patterns, traffic flow)
- Adding real-time traffic data to improve response time estimates
- Implementing a hierarchical reinforcement learning approach to handle multiple resource types
- Extending the model to optimize for multiple objectives (e.g., response time, cost, coverage)

6. Conclusion

This project demonstrated the effectiveness of reinforcement learning for optimizing emergency resource allocation in an urban setting. We implemented a custom environment based off accident data from Buffalo. By leveraging historical accident data from Buffalo, we developed a PPO-based agent that significantly outperformed random allocation baselines. The approach shows promising potential for real-world applications in emergency services management, potentially reducing response times and improving public safety outcomes.

References

1. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
2. Buffalo Open Data Portal. Received Traffic Incident Calls. https://data.buffalony.gov/Public-Safety/Received-Traffic-Incident-Calls/6at3-hpb5/about_data
3. Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. MIT press.
4. Konda, V. R., & Tsitsiklis, J. N. (2000). Actor-critic algorithms. In Advances in neural information processing systems (pp. 1008-1014).
5. Li, Y., Yu, R., Shahabi, C., & Liu, Y. (2018). Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. International Conference on Learning Representations (ICLR).

Contribution table

Project part	Team member	Contribution %

Dataset, cleanup, EDA, visualization	Sahil Sawant	40
	Fagun Patel	30
	Meghna Shekhar	30
Building PPO models, comparison of results	Sahil Sawant	30
	Fagun Patel	35
	Meghna Shekhar	35
Environment creation & reward structure	Sahil Sawant	33.33
	Fagun Patel	33.33
	Meghna Shekhar	33.33